

ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS (ESFOT)

SISTEMA DE GESTIÓN Y CONTROL PARA UNA PAPELERÍA

PROYECTO INTEGRADOR / TRABAJO FINAL DE LA ASIGNATURA DE PROGRAMACIÓN ORIENTADA A OBJETOS

AUTORES:

Leonel Alejandro Moreira Rengifo

leonel.moreira@epn.edu.ec

Melany Brillith Perugachi Toapanta

melany.perugachi@epn.edu.ec

DIRECTOR: ING. YADIRA GISSELA FRANCO ROCHA, MGS.

Quito, julio de 2026

Contenido

1. INTRODUCCIÓN	4
2. PROPÓSITO DE LA PRACTICA.....	4
3. OBEJTIVOS:.....	5
3.1 Objetivo general	5
3.2 Objetivos Específicos	5
4.DESARROLLO	6
4.1 Temática del proyecto	6
4.2 Problema que resuelve	7
4.3 Tecnologías utilizadas.....	7
4.4 Arquitectura del proyecto	7
4.5 Arquitectura y organización de los paquetes	9
4.6 Aplicación de los pilares de la Programación Orientada a Objetos	13
4.6.1 Encapsulamiento	13
4.6.2 Herencia	14
4.6.3 Abstracción.....	17
4.6.4 Polimorfismo.....	17
4.7 Diagrama de Clases (UML)	17
4.8 Base de datos	19
4.9 Gestión de Roles de Usuario	21
4.10 Interfaces del Sistema.....	25
4.10.1 Pantalla de Inicio de Sesión.....	25
4.10.2 Dashboard Principal	27
4.10.3 Gestión de Usuarios	28
4.10.4 Gestión de Productos	29
4.10.5 Gestión de Categorías.....	30
4.10.6 Gestión de Proveedores	30

4.10.7 Registro de Ventas	31
4.10.8 Reportes	32
4.10.9 Diseño de la Interfaz	33
4.11.1 Validación de campos obligatorios	34
4.11.2 Validación de datos numéricos	34
4.11.3 Validación de selección de registros	35
4.11.4 Confirmación de eliminación	35
5 CONCLUSIONES:	37
6. RECOMENDACIONES:.....	37
7. BIBLIOGRAFIA.....	38
8. ANEXOS	39
9. LINK DE VIDEO	44

1. INTRODUCCIÓN

El desarrollo de un sistema o software encaminado a los principios de la programación orientado a objetos constituye uno de los pilares fundamentales de la ingeniería e informática en el ámbito comercial moderno y organizado. La automatización de procesos como el control de inventarios, la gestión de ventas y el acceso restringido por roles permite a las pequeñas y medianas empresas contar con una solución que permita la administración de productos, reducir errores humanos, optimizar tiempos de atención y mantener la integridad de sus activos.

La asignatura Programación Orientada a Objetos promueve el diseño de soluciones informáticas además de fundamentos necesarios para el desarrollar app. Dentro de la formación académica de la Escuela de Formación de Tecnólogos (ESFOT) de la Escuela Politécnica Nacional (EPN), la asignatura de Programación Orientada a Objetos permite aplicar estos conocimientos en el desarrollo de soluciones informáticas que responden a necesidades reales.

Por esta razón el presente proyecto propone la creación de una Aplicación de Escritorio en JavaFX respaldada por una base de datos relacional para el control de inventario y punto de venta de una Papelería. La arquitectura implementa el patrón Data Access Object (DAO) y cuenta con un esquema de seguridad basado en roles (Administrador, Cajero y Bodeguero).

2. PROPÓSITO DE LA PRACTICA

Este proyecto tiene un propósito principal es integrar todo nuestro conocimiento adquirido en nuestro curso de POO dirigida por la Ing. ING. YADIRA GISSELA FRANCO ROCHA, MGS., cual nos ayudó a la integración de las herramientas

prácticas de POO (encapsulamiento, herencia, polimorfismo e interfaces) en un entorno real, mediante un lenguaje de JAVA y la biblioteca de JAVAFX.

Atraves de este proyecto se busca desarrollar una solución informática que permita mejorar la gestión de una papelería, facilitando el control de productos, categorías, proveedores, usuarios e inventario. También se pretende garantizar el rol que emplea cada trabajador de la papelería asumiendo su rol, la información es cargada en una base de datos (MySQL) así permitiendo un registro organizado y centralizado de cada operación que se puede realizar y la facilidad de la administración de recursos establecidos.

3. OBEJTIVOS:

3.1 Objetivo general

Desarrollar una aplicación de escritorio en JavaFX para la gestión integral de una papelería, que aplique de forma práctica los cuatro pilares de la Programación Orientada a Objetos (encapsulamiento, herencia, polimorfismo y abstracción) mediante una arquitectura en capas (modelo, controlador, DAO y view, db y app) conectada a una base de datos MySQL, con control de acceso diferenciado para tres roles de usuario.

3.2 Objetivos Específicos

1. Diseñar el modelo de dominio aplicando encapsulamiento y herencia, mediante una clase base (Persona) de la cual hereden las entidades que comparten atributos comunes (Usuario, Proveedor), evitando la duplicación de código y reflejando relaciones reales del negocio.
2. Implementar tres roles de usuario con permisos diferenciados con un sistema de autenticación (Administrador, Cajero, Bodeguero), de modo que cada uno acceda únicamente a las funciones que le corresponden según su responsabilidad dentro de la papelería.
3. Desarrollar una capa de acceso a datos (DAO) desacoplada de la lógica de interfaz, aplicando el principio de abstracción mediante una interfaz (ICRUD) que estandarice las operaciones de creación, consulta, actualización y eliminación de registros en la base de datos MySQL.
4. Aplicar polimorfismo mediante la implementación de la interfaz genérica ICRUD<T>, permitiendo que diferentes clases DAO (ProductoDAO,

UsuarioDAO, CategoriaDAO y ProveedorDAO) implementen los mismos métodos (insertar, actualizar, eliminar, listarTodos y buscarPorId), ejecutando una lógica específica para cada entidad.

4.DESARROLLO

4.1 Temática del proyecto

Sistema de Gestión de Papelería

El proyecto presente consiste en un Sistema de Gestión de Papelería, una app de escritorio implementada en JavaFX que permite administrar y gestionar procesos principales de una papelería, mediante una interfaz intuitiva con colores intuitivos y una base de datos MYSQL.

El sistema es diseñado para optimizar el control de la información que se relaciona con productos, categorías, proveedores, usuarios, ventas e inventario, todas estas informaciones se centran a una aplicación y reduciendo procesos manuales que pueden ocasionar errores, pérdida de información.

Además del manejo de datos se incorporó un mecanismo de autenticación basado en roles de usuario permitiendo que pueda acceder únicamente a sus funcionalidades y responsabilidades dentro de la empresa.

El sistema tiene principales funcionalidades del sistema como:

- Inicio de sesión mediante usuario y contraseña.
- Administración de usuarios.
- Gestión de productos.
- Administración de categorías.
- Gestión de proveedores.
- Registro de ventas.
- Control del inventario.
- Consulta de reportes.
- Acceso restringido según el rol del usuario.

El sistema se desarrolló y se implementó con los 4 pilares de POO, además de una arquitectura organizado por capas que facilitan el mantenimiento, escalabilidad y reutilización de código.

4.2 Problema que resuelve

En algunos barrios se encuentran muchas papelerías pequeñas que administran su inventario y ventas de forma manual o mediante hojas de cálculo, lo que esto puede ocasionar muchas veces inconvenientes como pérdida de información errores de registro de productos dificultades para controlar el stock y poca seguridad en acceso de datos.

El sistema desarrollado busca solucionar estos problemas mediante la app de escritorio que centraliza la administración de la información, permitiendo el registro de usuarios, controlar el inventario, gestionar proveedores, regitar ventas y consultar reportes de manera rápida, organizada y segura.

Finalmente, el sistema implementa un control de acceso basado en roles, evitando que un usuario pueda acceder a funciones que no le corresponden y mejorando la seguridad de la información almacenada.

4.3 Tecnologías utilizadas

Java versión 26

JavaFX

Maven

MySQL(Railway)

JDBC

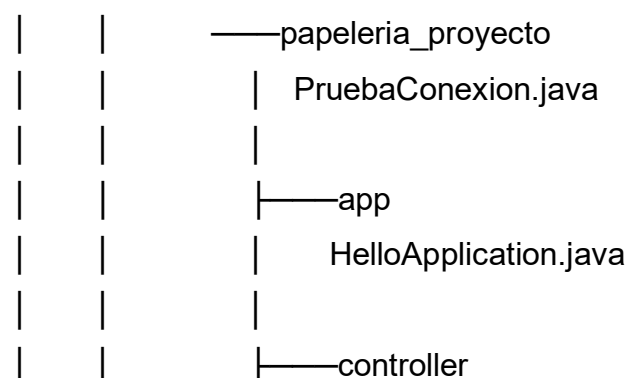
IntelliJ IDEA

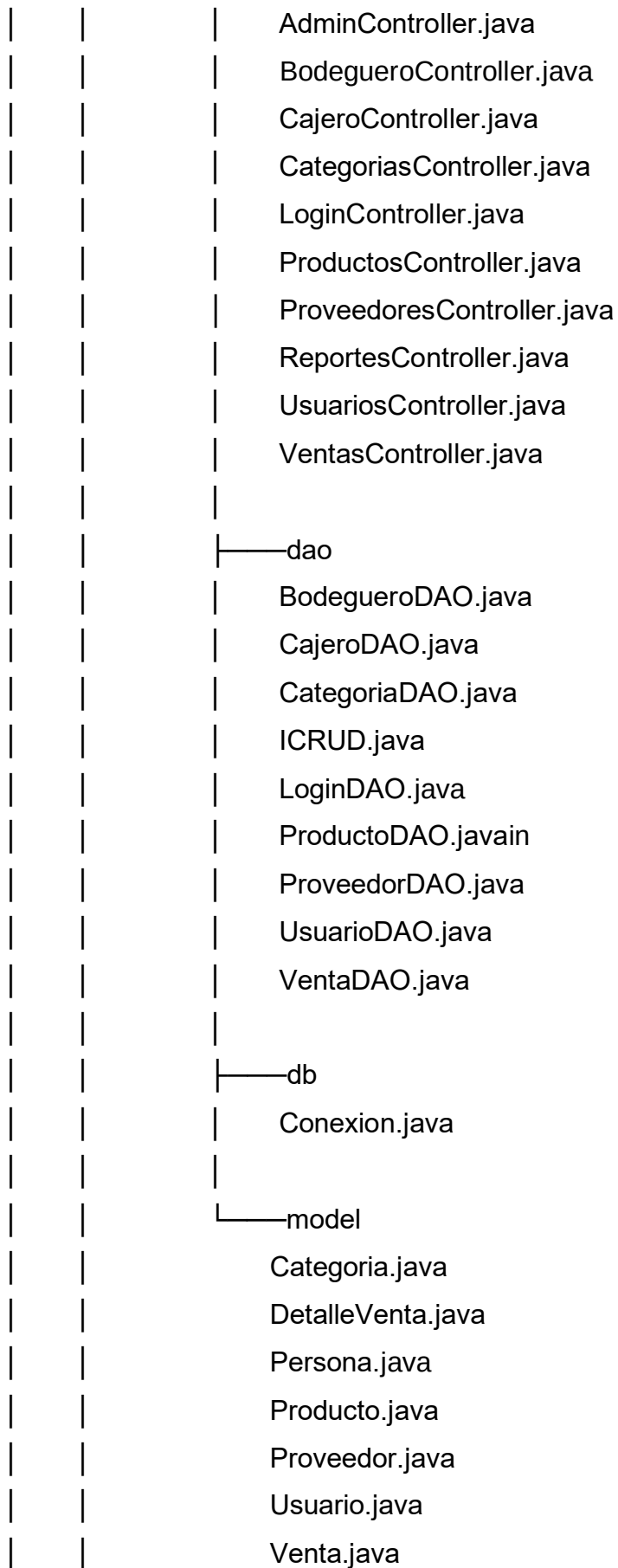
Scene Builder

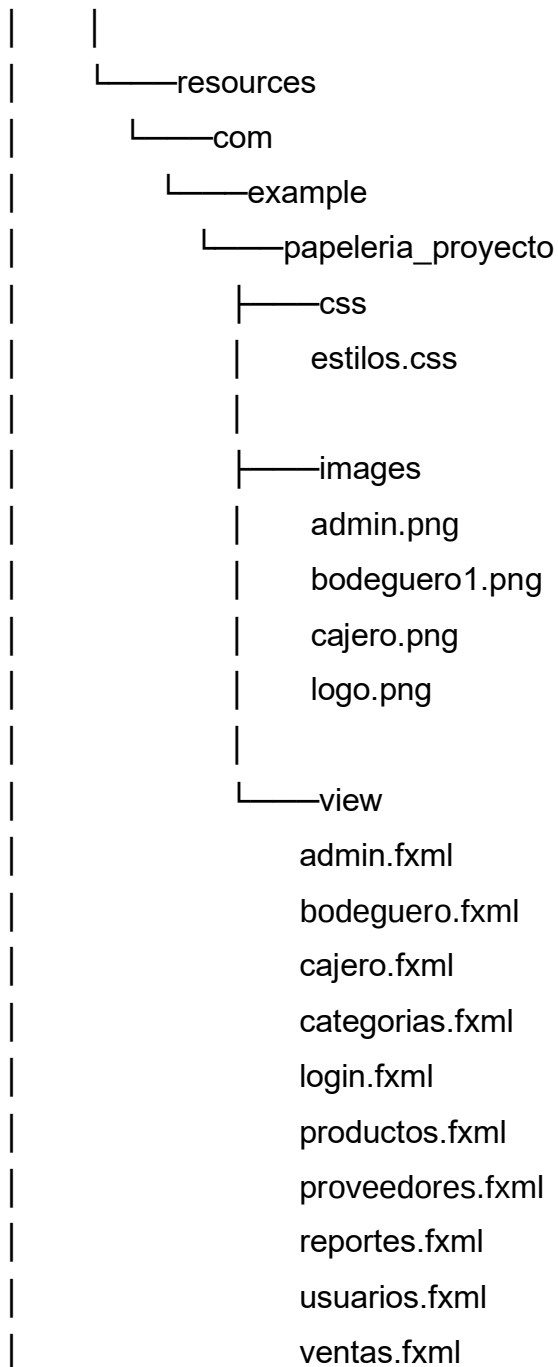
GitHub

4.4 Arquitectura del proyecto

El sistema desarrollado es organizado siguiendo una arquitectura en capas, la estructura principal del proyecto es la siguiente:







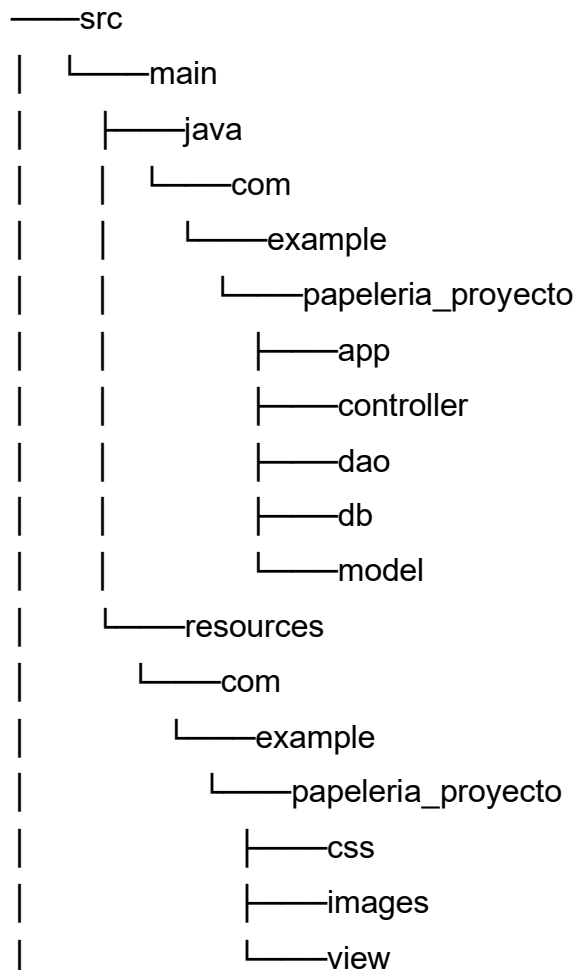
La organización permite separar claramente la interfaz gráfica, lógica del negocio, el acceso en la base de datos y entidades del sistema.

4.5 Arquitectura y organización de los paquetes

Se mantiene una organización con la estructura con el fin de facilitar cualquier mantenimiento del sistema, el proyecto fue desarrollado siguiendo una arquitectura

en capas, cada paquete cumple una responsabilidad específica dentro de la app, evitando mezclar la lógica del negocio con la interfaz gráfica.

Estructura principal:



- Paquete app

Contiene la clase principal HelloApplication, encargada de iniciar la ejecución del sistema.

- Paquete model

Contiene las clases que representan las entidades del negocio y almacenan la información utilizada por la aplicación

- Persona
- Usuario
- Producto
- Categoria

- Proveedor
- Venta
- DetalleVenta

Cada clase contiene atributos privados junto con sus respectivos métodos getter y setter, aplicando el principio de encapsulamiento.

Además, la clase Persona actúa como clase base, permitiendo que Usuario y Proveedor hereden los atributos comunes, aplicando el principio de herencia y evitando duplicar código.

- Paquete dao

Es responsable del acceso a la base de datos, contiene las consultas SQL necesarias para realizar operaciones CRUD sobre una entidad específica.

Las clases son

- UsuarioDAO
- ProductoDAO
- CategoriaDAO
- ProveedorDAO
- VentaDAO
- LoginDAO
- BodegueroDAO
- CajeroDAO

Se implementó la interfaz genérica ICRUD, la cual define los métodos comunes que deben implementar las clases encargadas de gestionar las entidades principales del sistema, el código demuestra la aplicación de los principios de abstracción y polimorfismo.

- Paquete controller

contiene la lógica que conecta la interfaz gráfica con la base de datos.

Cada controlador responde a las acciones realizadas por el usuario sobre la interfaz, valida la información ingresada y utiliza las clases DAO para ejecutar las operaciones correspondientes.

Los principales controladores son:

- LoginController
- AdminController
- UsuariosController
- ProductosController
- CategoriasController
- ProveedoresController
- VentasController
- ReportesController
- CajeroController
- BodegueroController

- Paquete db

Contiene la clase Conexión, responsable de establecer la comunicación entre la aplicación y la base de datos MySQL mediante JDBC.

Sus principales funciones son:

- Configurar los parámetros de conexión.
- Abrir conexiones con la base de datos.
- Proporcionar una conexión reutilizable para los DAO.

- Carpeta View

contiene los archivos FXML, los cuales representan las interfaces gráficas de la aplicación.

Entre las principales vistas se encuentran:

login.fxml

admin.fxml

usuarios.fxml

productos.fxml

categorias.fxml

proveedores.fxml

ventas.fxml

reportes.fxml

cajero.fxml

Bodeguero.fxml

- Carpeta css

Contiene la hoja de estilos que se utilizó para personalizar la apariencia de la app para que sea agradable para el usuario.

- Carpeta images

Contiene las imágenes que contiene los recursos gráficos utilizados por el sistema como logotipos de cada dashboard según el rol.

4.6 Aplicación de los pilares de la Programación Orientada a Objetos

La Programación Orientada a Objetos (POO) constituye la base del desarrollo de este proyecto. Durante la implementación del sistema de gestión de papelería se aplicaron los cuatro pilares fundamentales de la POO: encapsulamiento, herencia, abstracción y polimorfismo. Estos principios permitieron desarrollar una aplicación organizada, reutilizable, escalable y fácil de mantener.

4.6.1 Encapsulamiento

El encapsulamiento consiste en proteger la información de una clase, evitando que sus atributos sean modificados directamente desde otras clases. Para ello, los atributos fueron declarados con acceso `private`, permitiendo su manipulación únicamente mediante métodos públicos (getters y setters).

En el proyecto, este principio se evidencia en todas las clases del paquete `model`, como `Producto`, `Usuario`, `Proveedor`, `Categoría` y `Venta`.

```

public class Producto { 37 usages LeitoM10 *
    private int idProducto; 3 usages
    private String codigo; 3 usages
    private String nombre; 4 usages
    private String descripcion; 3 usages
    private BigDecimal precio; 3 usages
    private int stock; 3 usages
    private int estado; 3 usages
    private int idCategoria; 3 usages

    public Producto() {}

    public Producto(int idProducto, String codigo, String nombre, String descripcion, 9 usages
        BigDecimal precio, int stock, int estado, int idCategoria) {
        this.idProducto = idProducto;
        this.codigo = codigo;
        this.nombre = nombre;
        this.descripcion = descripcion;
        this.precio = precio;
        this.stock = stock;
        this.estado = estado;
        this.idCategoria = idCategoria;
    }

    public Producto(String codigo, String nombre, String descripcion, 6 usages LeitoM10
        BigDecimal precio, int stock, int estado, int idCategoria) {
        this(idProducto: 0, codigo, nombre, descripcion, precio, stock, estado, idCategoria);
    }

    public int getIdProducto() { return idProducto; }
    public void setIdProducto(int idProducto) { this.idProducto = idProducto; }
    public String getCodigo() { return codigo; }
    public void setCodigo(String codigo) { this.codigo = codigo; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getDescripcion() { return descripcion; }
    public void setDescripcion(String descripcion) { this.descripcion = descripcion; }
    public BigDecimal getPrecio() { return precio; }
    public void setPrecio(BigDecimal precio) { this.precio = precio; }
    public int getStock() { return stock; }
    public void setStock(int stock) { this.stock = stock; }
    public int getEstado() { return estado; }
    public void setEstado(int estado) { this.estado = estado; }
    public int getIdCategoria() { return idCategoria; }
    public void setIdCategoria(int idCategoria) { this.idCategoria = idCategoria; }
    @Override LeitoM10
    public String toString() { return nombre; }
}

```

Encapsulamiento aplicado en la clase Producto 1

4.6.2 Herencia

La herencia permite reutilizar atributos y métodos comunes entre varias clases mediante una relación padre-hijo.

En el sistema se creó la clase Persona, la cual agrupa atributos comunes como nombre, teléfono y correo. Posteriormente, las clases Usuario y Proveedor heredan dichos atributos utilizando la palabra reservada extends.

```

1 package com.example.papeleria_proyecto.model;
2
3 public class Persona { 2 usages 2 inheritors Melany200415
4
5     protected String nombre;
6     protected String telefono; 3 usages
7     protected String correo; 3 usages
8
9     public Persona() { no usages Melany200415
10    }
11
12    public Persona(String nombre, String telefono, String correo) { 3 usages
13        this.nombre = nombre;
14        this.telefono = telefono;
15        this.correo = correo;
16    }
17
18    > public String getNombre() { return nombre; }
19
20    > public void setNombre(String nombre) { this.nombre = nombre; }
21
22    > public String getTelefono() { return telefono; }
23
24    > public void setTelefono(String telefono) { this.telefono = telefono; }
25
26    > public String getCorreo() { return correo; }
27
28    > public void setCorreo(String correo) { this.correo = correo; }
29
30 }

```

Clase padre 1

```

public class Usuario extends Persona { 28 usages  👤 LeitoM1

    private int idUsuario; 3 usages
    private String apellido; 4 usages
    private String usuario; 3 usages
    private String contrasena; 3 usages
    private int estado; 4 usages
    private int idRol; 3 usages

    public Usuario( 7 usages  👤 Melany200415 +1
        int idUsuario,
        String nombre,
        String apellido,
        String usuario,
        String contrasena,
        String telefono,
        String correo,
        int estado,
        int idRol
    ) {

        super(nombre, telefono, correo);

        this.idUsuario = idUsuario;
        this.apellido = apellido;
        this.usuario = usuario;
        this.contrasena = contrasena;
        this.estado = estado;
        this.idRol = idRol;
    }

    public Usuario( 5 usages  👤 Melany200415
        String nombre,
        String apellido,
        String usuario,
        String contrasena,
        String telefono,
        String correo,
        int estado,
        int idRol
    ) {

        this(
            idUsuario: 0,
            nombre,
            apellido,
            usuario,
            contrasena,
            telefono,
            correo,

```

Clase Hija 1

4.6.3 Abstracción

La abstracción consiste en definir únicamente las operaciones necesarias sin especificar inicialmente su implementación.

Este principio fue aplicado mediante la interfaz genérica ICRUD, la cual establece las operaciones básicas que cualquier clase DAO debe implementar para administrar la información de la base de datos. La interfaz define que métodos deben existir, mientras que las clases DAO implementa su lógica correspondiente en cada método.

```
1 package com.example.papeleria_proyecto.dao;
2
3 import javafx.collections.ObservableList;
4
5 public interface ICRUD<T> { 4 usages 4 implementations Melany200415
6
7     boolean insertar(T objeto); 4 implementations Melany200415
8
9     boolean actualizar(T objeto); 4 usages 4 implementations Melany200415
10
11     boolean eliminar(int id); 4 implementations Melany200415
12
13     ObservableList<T> listarTodos(); 10 usages 4 implementations Melany200415
14 }
```

Interfaz ICRUD utilizada como el mecanismo de abstracción

4.6.4 Polimorfismo

El polimorfismo permite que diferentes clases respondan a un mismo conjunto de métodos de forma distinta.

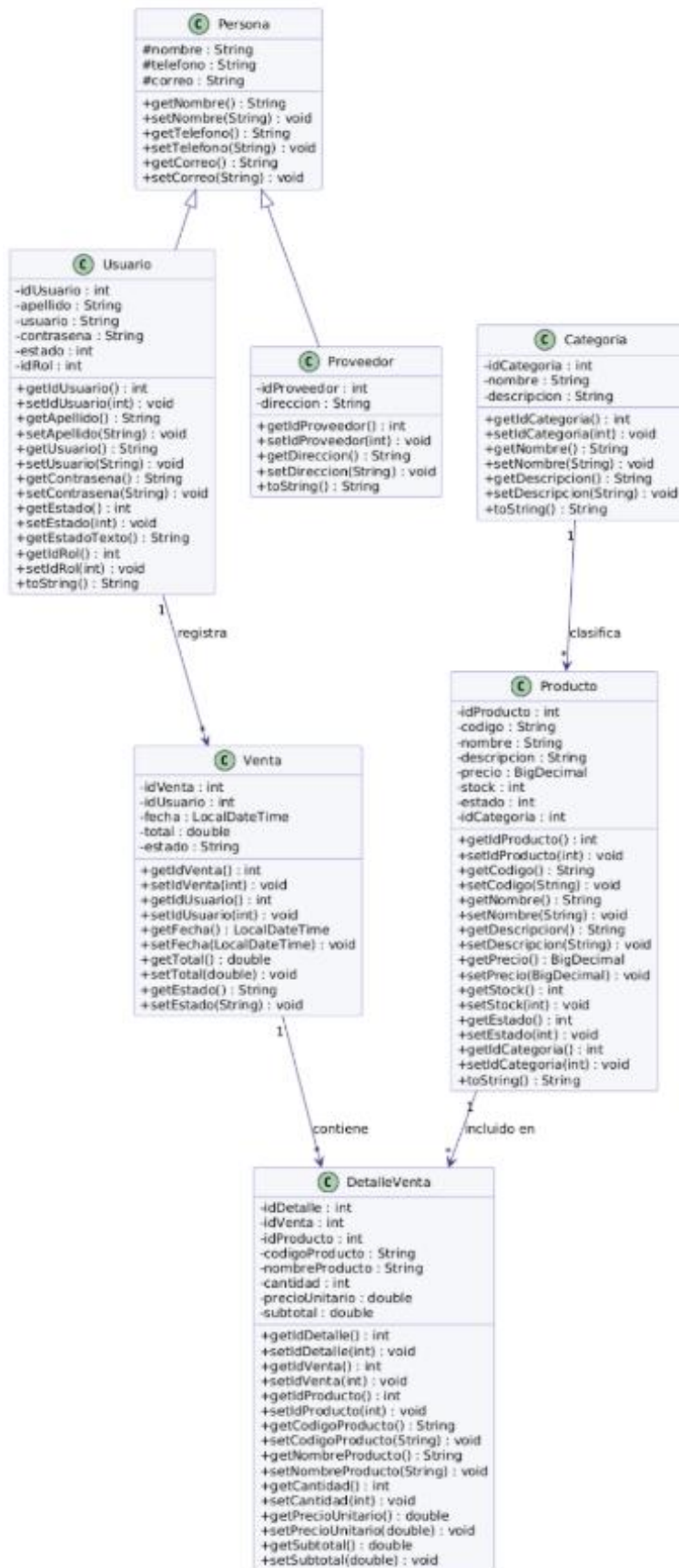
```
public class ProductoDAO implements ICRUD<Producto> { 9 u
```

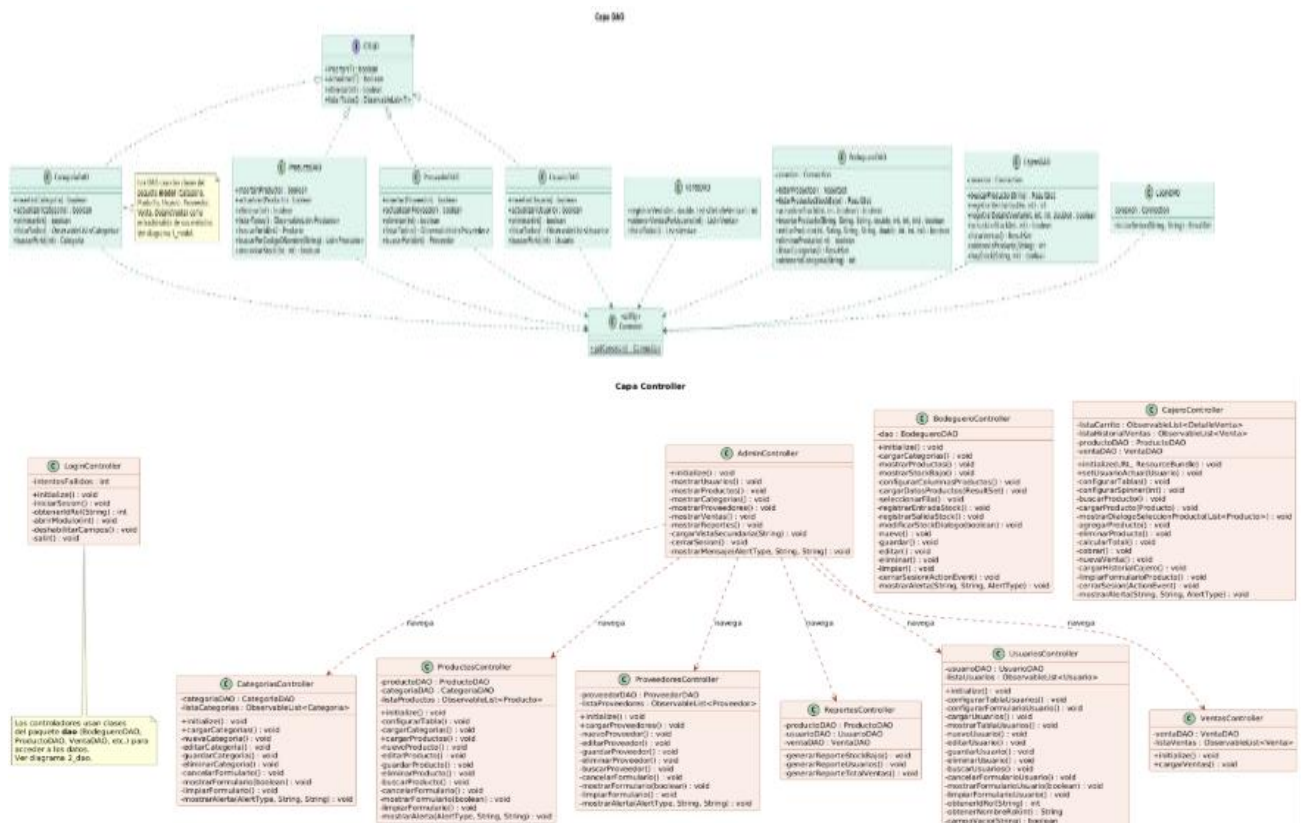
Implementación de la interfaz ICRUD en la clase Producto ejemplo

4.7 Diagrama de Clases (UML)

El diagrama de clases permitió definir claramente las responsabilidades de cada clase antes de la implementación del sistema. Además, facilitó la organización del proyecto siguiendo una arquitectura orientada a objetos, favoreciendo la reutilización del código, el modularidad y el mantenimiento de la aplicación.

Capa Model



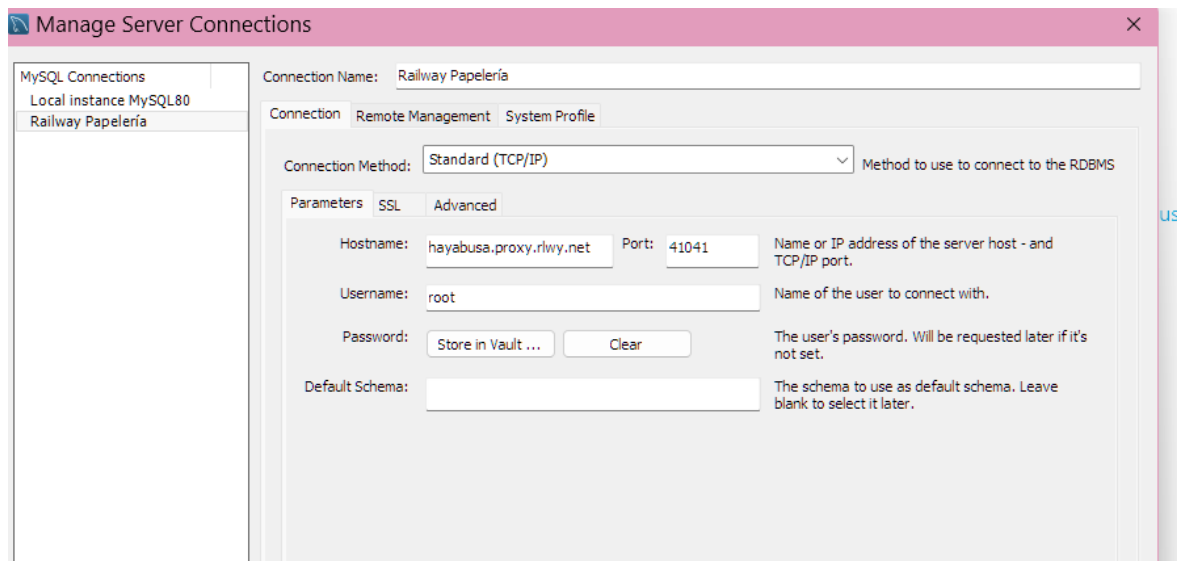


4.8 Base de datos

Para el almacenamiento de la información, el sistema utiliza una base de datos MySQL alojada en la nube mediante Railway. Esta plataforma permitió disponer de una base de datos remota, accesible desde cualquier equipo con conexión a Internet, eliminando la necesidad de instalar un servidor MySQL local en cada computadora donde se ejecuta la aplicación.

Railway proporciona el servicio de alojamiento de la base de datos, mientras que MySQL actúa como el sistema gestor de bases de datos relacional encargado de administrar toda la información del sistema.

La aplicación JavaFX establece la comunicación con la base de datos utilizando JDBC (Java Database Connectivity) mediante la clase Conexión, ubicada en el paquete db, la cual centraliza todas las medidas necesarias para realizar la conexión.



Vinculación de Railway y Mysql

```
package com.example.papeleria_proyecto.db;

import ...

public class Conexion {

    private static final String URL = "jdbc:mysql://hayabusa.proxy.rlwy.net:41041/papeleria?useSSL=false&serverTimezone=UTC";

    private static final String USUARIO = "root";

    private static final String PASSWORD = "kkpPLTfoygJzQYNzhfStiDmKqpBkSbLG";

    public static Connection getConexion() {
        try {
            Connection conexion = DriverManager.getConnection(URL, USUARIO, PASSWORD);
            System.out.println("Conexión exitosa a la base de datos.");
            return conexion;
        } catch (SQLException e) {
            System.out.println("Error al conectar: " + e.getMessage());
            return null;
        }
    }
}
```

Clase conexion 1

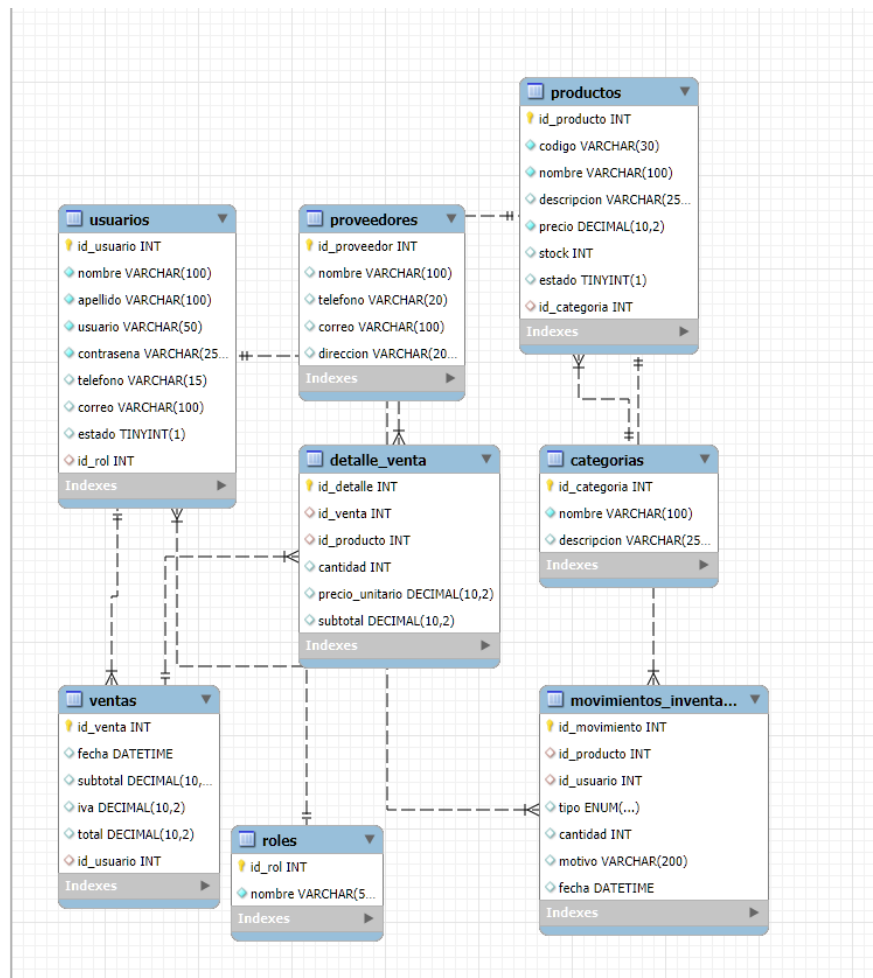
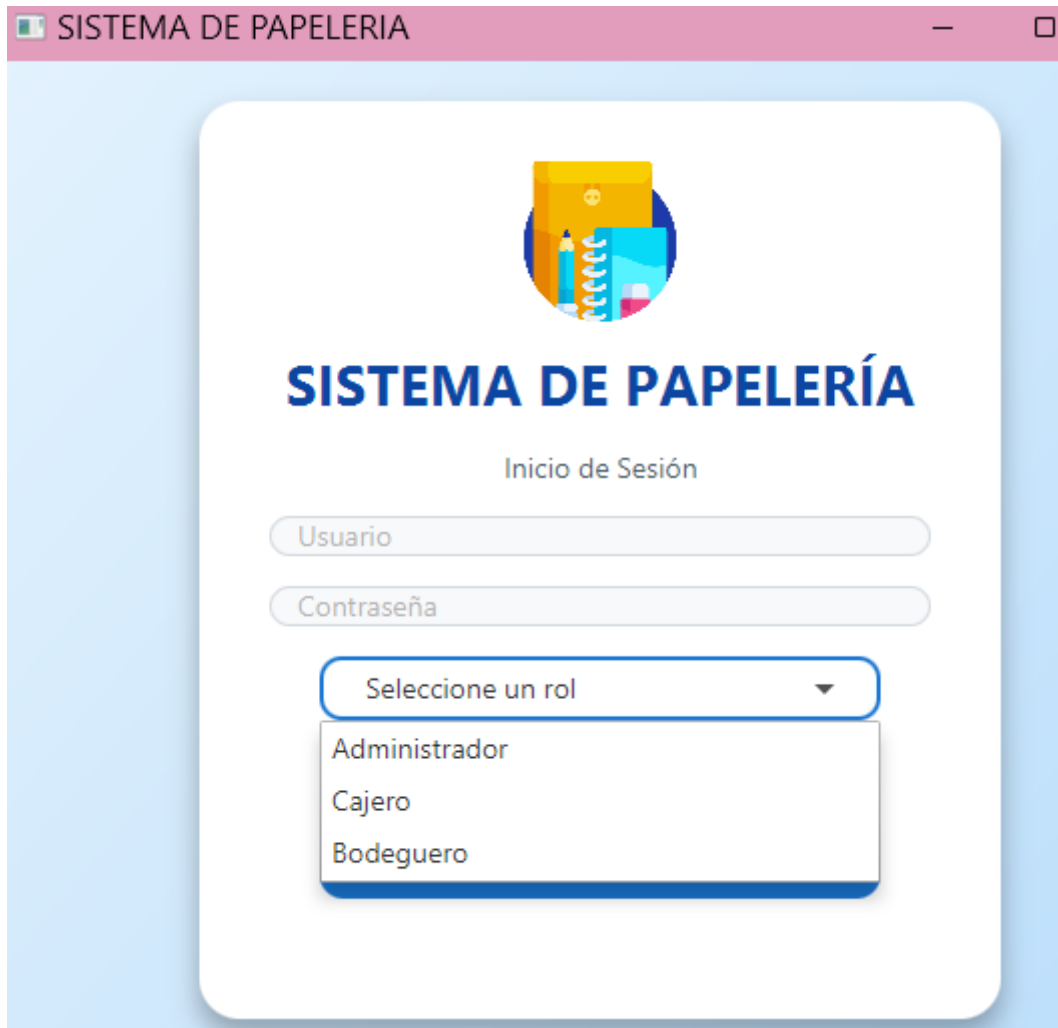


Diagrama de tablas y relaciones en MySQL 1

4.9 Gestión de Roles de Usuario

El sistema implementa la autenticación basada en roles con el objetivo de restringir el acceso a determinadas funcionalidades según las responsabilidades de cada usuario dentro de la papelería. De esta manera, cada persona puede acceder únicamente a los módulos necesarios para realizar su trabajo, mejorando la seguridad y organización del sistema.

El proceso de autenticación se realiza mediante un formulario de inicio de sesión, donde el usuario ingresa su nombre de usuario y contraseña. Una vez verificadas las credenciales en la base de datos MySQL alojada en Railway, el sistema identifica el rol asignado y carga la interfaz correspondiente.



SISTEMA DE PAPELERIA

SISTEMA DE PAPELERÍA

Inicio de Sesión

Usuario

Contraseña

Seleccione un rol

- Administrador
- Cajero
- Bodeguero

Login 1

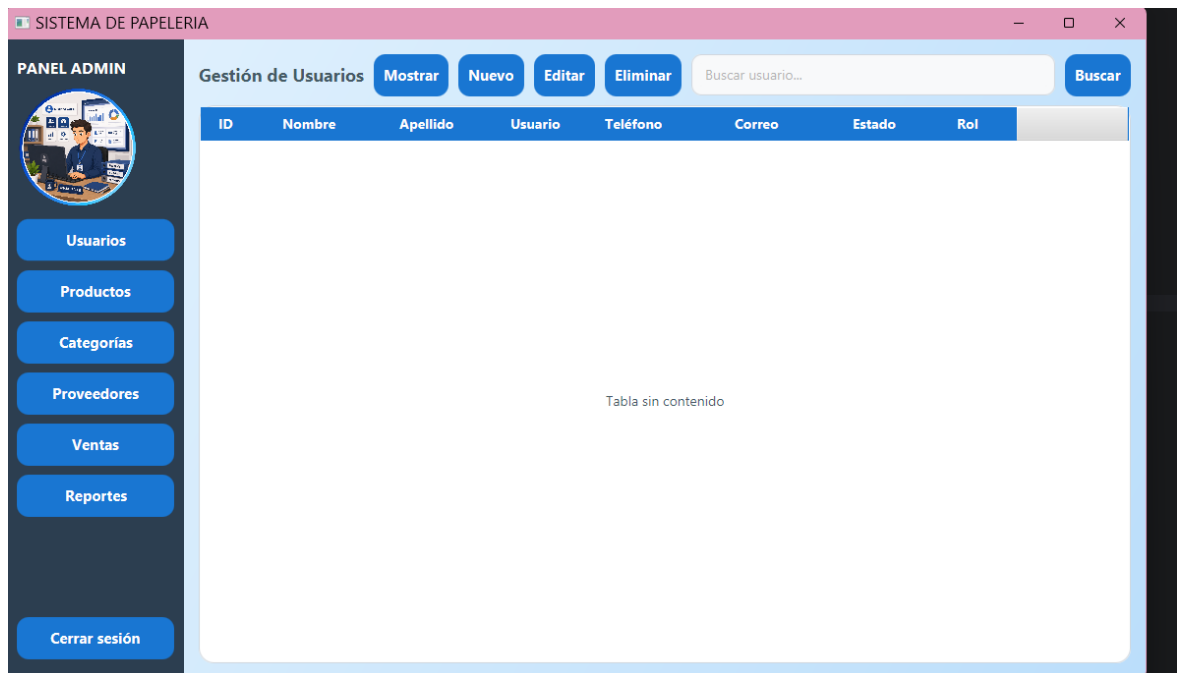
4.9.1 Rol Administrador

El Administrador dispone de acceso de mayor nivel de permisos del sistema, siendo el responsable de la administración general de la papelería.

Entre sus principales funciones se encuentran:

- Administrar usuarios.
- Registrar, modificar y eliminar productos.
- Gestionar categorías.
- Registrar proveedores.
- Consultar reportes.

- Supervisar el inventario.



Dashboard Admin

4.9.2 Rol Cajero

El Cajero es el encargado de realizar las operaciones relacionadas con las ventas.

Sus funciones principales son:

- Registrar ventas.
- Consultar productos disponibles.
- Buscar productos.
- Visualizar información necesaria para realizar la venta.

El cajero no puede acceder a los módulos administrativos.

SISTEMA DE PAPELERIA



MÓDULO CAJERO

Registro y Gestión de Ventas

Cerrar Sesión

Buscar Por:

Buscar

Producto: Stock Disponible: Precio (\$):

Cantidad:

+ Agregar al...

Nueva Venta

Eliminar Ite...

Cobrar e I...

Carrito de Venta Actual

Código	Producto	Cantidad	Precio Unitario	Subtotal
Tabla sin contenido				

TOTAL A PAGAR: 0.00

Mis Ventas Realizadas (Historial Cajero)

Nº Factura / Venta	Fecha y Hora	Total (\$)	Estado
Tabla sin contenido			

Dashboard Cajero

4.9.3 Rol Bodeguero

El Bodeguero es responsable del control del inventario y de la administración de los productos almacenados.

Entre sus funciones se encuentran:

- Consultar el inventario.
- Registrar entradas de productos.
- Actualizar existencias.
- Verificar el stock disponible.
- Consultar información relacionada con los proveedores.



MÓDULO BODEGUERO

Control de Inventario y Stock

Ver Productos

Stock Bajo

Categorías

Cerrar Sesión

Código:

Precio (\$):

Categoría:

Nombre:

Stock:

Estado:

Descripción:

Nuevo
Guardar
Editar
Eliminar
Entrada Stock
Salida (Mer...
Limpiar

Código	Nombre	Descripción	Precio	Stock	Estado	Categoría
E10H	Hojas papel		0.12	50	Activo	Papelería
E50H	laminas		1.50	185	Activo	Otros
E40H	LAPIZ	LAPIZ NORICA	0.75	6	Activo	Papelería
E676	PIZZA	Rica	12.00	2	Inactivo	Otros

Dashboard Bodeguero

4.10 Interfaces del Sistema

La aplicación fue desarrollada utilizando JavaFX, empleando archivos FXML para el diseño de las interfaces gráficas y hojas de estilo CSS para mejorar la apariencia visual. Cada ventana fue diseñada con el objetivo de facilitar la interacción del usuario y permitir una navegación intuitiva entre los diferentes módulos del sistema. Tenemos:

4.10.1 Pantalla de Inicio de Sesión

La pantalla de inicio de sesión constituye el punto de acceso al sistema. En ella, el usuario debe ingresar su nombre de usuario y contraseña para que el sistema valide sus credenciales y determine el rol asignado.

Una vez autenticado correctamente, la aplicación carga automáticamente el panel correspondiente al rol del usuario.

Funciones principales:

- Ingreso de usuario y contraseña.

- Validación de credenciales.
- Identificación del rol.
- Redirección al dashboard correspondiente.
- Mensajes de error cuando las credenciales son incorrectas.



SISTEMA DE PAPELERÍA

Inicio de Sesión

Seleccione un rol



Iniciar Sesión

Salir

Complete todos los campos.



SISTEMA DE PAPELERÍA

Inicio de Sesión

Administrador ▼

Iniciar Sesión

Salir

Credenciales incorrectas. Intentos restantes: 1

4.10.2 Dashboard Principal

El dashboard constituye la ventana principal de la aplicación y funciona como centro de navegación hacia los diferentes módulos del sistema.

Dependiendo del rol del usuario, el sistema habilita únicamente las opciones que le corresponden.

Desde este panel es posible acceder a los módulos de administración, ventas, inventario y reportes.



4.10.3 Gestión de Usuarios

Esta interfaz permite administrar los usuarios registrados en el sistema.

Las principales operaciones disponibles son:

- Registrar nuevos usuarios.
- Consultar usuarios existentes.
- Modificar información.
- Eliminar registros.
- Buscar usuarios.

Los datos se muestran mediante un componente TableView, facilitando la visualización de la información almacenada.

PANEL ADMIN



Usuarios

Productos

Categorías

Proveedores

Ventas

Reportes

Gestión de Usuarios

Mostrar

Nuevo

Editar

Eliminar

Buscar usuario...

Buscar

ID	Nombre	Apellido	Usuario	Teléfono	Correo	Estado
1	Leonel	Moreira	Leito10	0999999999	leo@gmail.com	Activo
2	Melany	Perugachi	Mela	0978824529	melanybrillith8@...	Activo
3	Carlos	Pérez	bodeguero	0987654321	bodeguero@pap...	Activo
4	Juan	Guitierrez	admin	0991234567	admin@papeleri...	Activo
5	Juan	Rodriguez	Juan123	0999999999	juan@gmail.com	Activo
6	Leo	Moreira	leito	0888888888	leo@epn	Activo
7	Brillith	Toapanta	bodega2	0977777777	bodega2@gmail....	Activo
8	alexandra	toapanta	cajero2	09785463...	caja2@gmail.com	Inactivo
9	Jonathan	Cadena	cajero 3	0978585856	caja3@gmail.com	Activo
10	Juan	Perez	Junito.perez	0999999999	juan.perez@gmi...	Activo
11	Pepe	Moreira	ale10	0999999999	al@gmail.com	Inactivo
12	Alejandro	Moreira	ale123	0999999999	aleo@gmail.com	Activo
13	Teresa	Molina	tere123	0999999999	tere@gmail.com	Activo
14	Yadira	Franco	Yadi12	0888888888	yadi@gmail.com	Activo

Nuevo Usuario

Nombre

Apellido

Usuario

Contraseña

Teléfono

Correo

Administrador

Activo

Guardar

Cancelar

4.10.4 Gestión de Productos

Esta interfaz permite administrar los productos comercializados por la papelería.

Entre sus funciones se encuentran:

- Registrar productos.
- Editar información.
- Actualizar el stock.
- Eliminar productos.
- Buscar productos.
- Visualizar el inventario.

PANEL ADMIN



Usuarios

Productos

Categorías

Proveedores

Ventas

Reportes

Gestión de Productos

Mostrar Todos

Nuevo

Editar

Desactivar/Eliminar

Buscar por código o nombre...

Buscar

ID	Código	Nombre	Descripción	Precio (\$)	Stock	Estado	Categ
1	E40H	LAPIZ	LAPIZ NORICA	0.75	6	Activo	3
4	E50H	laminas		1.50	185	Activo	5
5	E10H	Hojas papel		0.12	50	Activo	3

Nuevo Producto

Código del producto

Nombre del Producto

Descripción

Precio (Ej: 12.50)

Stock Inicial

Activo

Seleccionar Categoría

Guardar

Cancelar

4.10.5 Gestión de Categorías

El módulo de categorías permite organizar los productos según su tipo.

Las principales operaciones son:

- Registrar categorías.
- Modificar categorías.
- Eliminar categorías.
- Consultar categorías existentes.

The screenshot displays the 'Gestión de Categorías' (Category Management) interface. On the left is a sidebar with navigation links: 'Usuarios', 'Productos', 'Categorías', and 'Proveedores'. The main area has a header with 'Gestión de Categorías' and buttons for 'Mostrar Todas', 'Nueva', 'Editar', and 'Eliminar'. Below this is a table of categories:

ID	Nombre Categoría	Descripción
2	Suministros de Oficina	
3	Papelería	Productos de suministro
4	Tecnología	
5	Otros	
7	Útiles Escolares	
8	Suministros de Oficina	
10	Tecnología	
11	Otros	

To the right of the table is a 'Nueva Categoría' (New Category) form with two input fields: 'Nombre de la categoría' and 'Descripción detallada'. At the bottom of the form are 'Guardar' (Save) and 'Cancelar' (Cancel) buttons.

4.10.6 Gestión de Proveedores

Este módulo administra la información de los proveedores encargados de abastecer la papelería.

Permite:

- Registrar proveedores.
- Actualizar información.
- Consultar registros.
- Eliminar proveedores.



MÓDULO CAJERO

Registro y Gestión de Ventas

Cerrar Sesión

Buscar Por:

Buscar

Producto: Stock Disponible: Precio (\$):

Cantidad:

+ Agregar al...

Nueva Venta

Eliminar It...

Cobrar e I...

Carrito de Venta Actual

Código	Producto	Cantidad	Precio Unitario	Subtotal	
E50H	laminas	1	1.5	1.5	
E40H	LAPIZ	1	0.75	0.75	
E10H	Hojas papel	3	0.12	0.36	

TOTAL A PAGAR: \$2.61

Mis Ventas Realizadas (Historial Cajero)

N° Factura / Venta	Fecha y Hora	Total (\$)	Estado	

También ver: [consultas](#)

4.10.8 Reportes

El módulo de reportes permite consultar información almacenada en la base de datos para facilitar el análisis y seguimiento de las operaciones del sistema.

Entre las consultas disponibles se encuentran:

- Productos registrados.
- Inventario disponible.
- Ventas realizadas.
- Información de usuarios.
- Información de proveedores.



MÓDULO BODEGUERO

Control de Inventario y Stock

Ver Productos

Stock Bajo

Categorías

Cerrar Sesión

Código

Nombre

Precio (\$)

Stock

Categoría

▼

Estado

▼

Descripción

Descripción del producto

Nuevo
Guardar
Editar
Eliminar
Entrada Stock
Salida (Mer...
Limpiar

Código	Nombre	Descripción	Precio	Stock	Estado	Categoría
E10H	Hojas papel		0.12	47	Activo	Papelería
E50H	laminas		1.50	184	Activo	Otros
E40H	LAPIZ	LAPIZ NORICA	0.75	4	Activo	Papelería
E676	PIZZA	Rica	12.00	2	Inactivo	Otros

4.10.9 Diseño de la Interfaz

Para mejorar la experiencia del usuario se aplicó una hoja de estilos CSS, logrando una interfaz uniforme y fácil de utilizar.

Entre las características del diseño se destacan:

- Colores consistentes en todas las ventanas.
- Botones con estilos personalizados.
- Organización clara de los controles.
- Uso de íconos e imágenes.
- Distribución adecuada de los componentes.

El uso de JavaFX junto con CSS permitió desarrollar una interfaz moderna, intuitiva y adaptable a los diferentes módulos del sistema.

4.11 Validaciones del Sistema

4.11.1 Validación de campos obligatorios

Antes de guardar un registro, el sistema verifica que todos los campos obligatorios hayan sido completados por el usuario.

Si algún campo se encuentra vacío, la operación es cancelada y se muestra un mensaje de advertencia mediante un cuadro de diálogo (Alert).



The image shows a login interface for a system titled "SISTEMA DE PAPELERÍA". At the top is a circular logo containing icons of stationery items like a yellow folder, a blue notepad, a green pencil, and a pink eraser. Below the logo, the title "SISTEMA DE PAPELERÍA" is displayed in a large, bold, blue font. Underneath the title, the text "Inicio de Sesión" is centered. The form consists of three input fields: "Usuario", "Contraseña", and a dropdown menu labeled "Seleccione un rol". Below these fields are two prominent blue buttons: "Iniciar Sesión" and "Salir". At the bottom of the form, a message "Complete todos los campos." is displayed.

4.11.2 Validación de datos numéricos

Los campos que representan valores numéricos, como precio, cantidad o stock, son verificados para impedir el ingreso de datos inválidos.

El sistema únicamente acepta valores numéricos positivos, evitando errores durante los cálculos y el almacenamiento de la información.

MÓDULO BORRADOR
Control de Inventario

Código: E50H Nombre: Precio (\$): 1.50 Stock: Categoría: Otros Estado: Activo

Descripción del producto

Nuevo **Guardar** **Editar** **Eliminar** **Entrada Stock** **Salida (Mer...** **Limpiar**

Código	Nombre	Descripción	Precio	Stock	Estado	Categoría
E10H	Hojas papel		0.12	47	Activo	Papelería
E50H	laminas		1.50	184	Activo	Otros

Error
Ingrese una cantidad entera válida mayor a 0.
Aceptar

4.11.3 Validación de selección de registros

Para las operaciones de edición y eliminación, el sistema verifica que el usuario haya seleccionado previamente un registro de la tabla.

En caso contrario, se muestra un mensaje indicando que debe seleccionarse un elemento antes de continuar.

MÓDULO BORRADOR
Control de Inventario

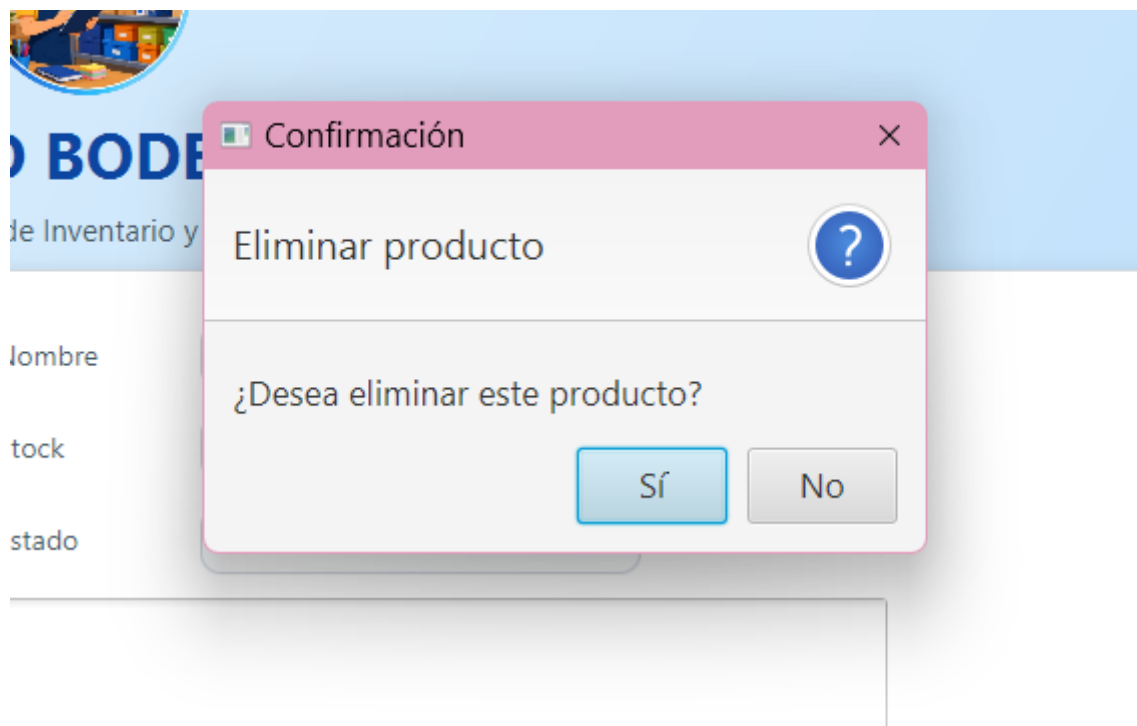
Código: Nombre: Precio (\$): 0.00 Stock: Categoría: Estado:

Atención
Seleccione un producto de la tabla.
Aceptar

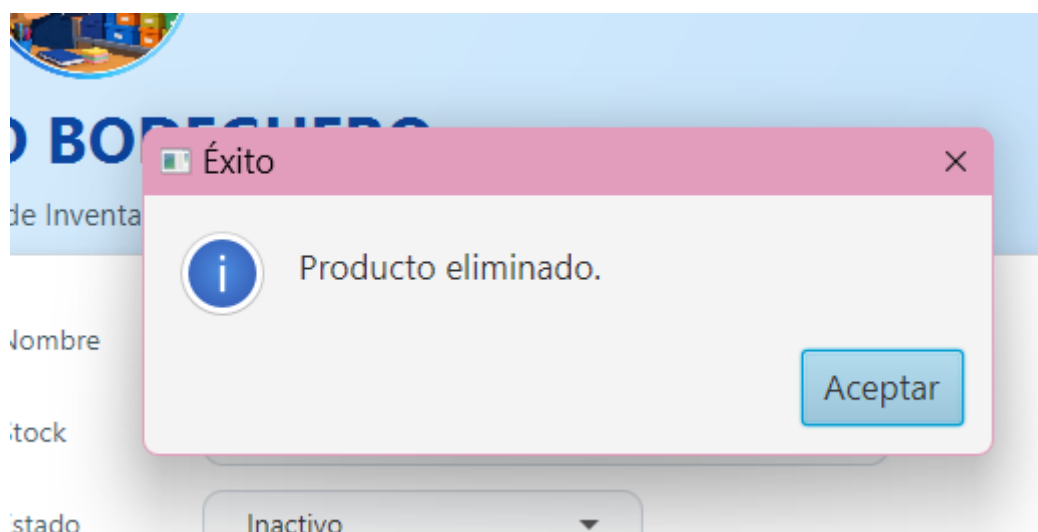
4.11.4 Confirmación de eliminación

Antes de eliminar un registro, la aplicación solicita una confirmación al usuario para evitar pérdidas accidentales de información.

Solo si el usuario confirma la operación, el registro es eliminado de la base de datos



Descripción	Precio	Stock	Estado	Categoría
	0.12	47	Activo	Papelería
	1.50	184	Activo	Otros
Z NORICA	0.75	9	Activo	Papelería
	12.00	1	Inactivo	Otros



5 CONCLUSIONES:

1. El desarrollo del sistema de gestión para papelería permitió resolver de manera eficiente los procesos de inventario, ventas y administración de usuarios, reemplazando métodos manuales por una solución informática más rápida, organizada y confiable.
2. Durante la implementación se aplicaron los principios fundamentales de la Programación Orientada a Objetos. El encapsulamiento permitió proteger la información de las clases, la herencia se utilizó mediante la clase Persona, la abstracción facilitó la definición de comportamientos comunes y el polimorfismo se implementó a través de la interfaz ICRUD, logrando un código reutilizable y fácil de mantener.
3. La integración de JavaFX, MySQL y Railway permitió construir una aplicación funcional con acceso remoto a los datos, mientras que la arquitectura por capas y el uso de GitHub favorecieron una adecuada separación de responsabilidades, el trabajo colaborativo y la escalabilidad del sistema.

6. RECOMENDACIONES:

Para futuras versiones del sistema, sería recomendable fortalecer la seguridad mediante la implementación de algoritmos de cifrado más robustos para el almacenamiento de contraseñas, además del uso de conexiones seguras y mecanismos adicionales de autenticación que protejan la información de los usuarios y de la base de datos.

Asimismo, se podría ampliar el módulo de reportes incorporando la exportación de información a formatos como PDF y Excel, además de generar estadísticas y gráficos de ventas que faciliten la toma de decisiones por parte del administrador del negocio.

Otra mejora importante sería implementar un sistema de respaldo automático de la base de datos, permitiendo recuperar la información ante posibles fallos técnicos o pérdidas de datos y garantizando una mayor disponibilidad del sistema.

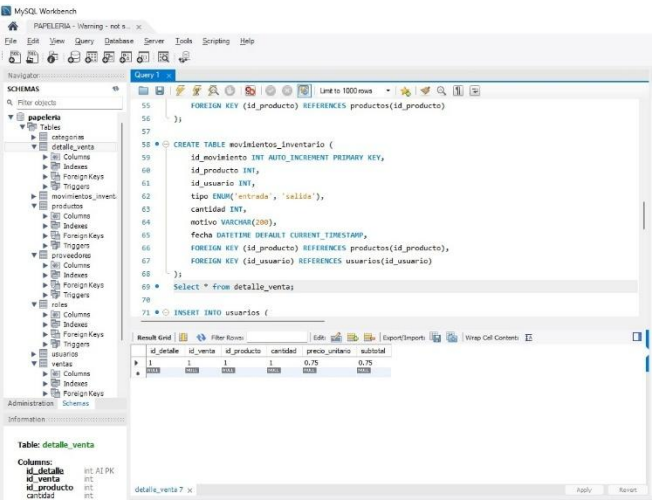
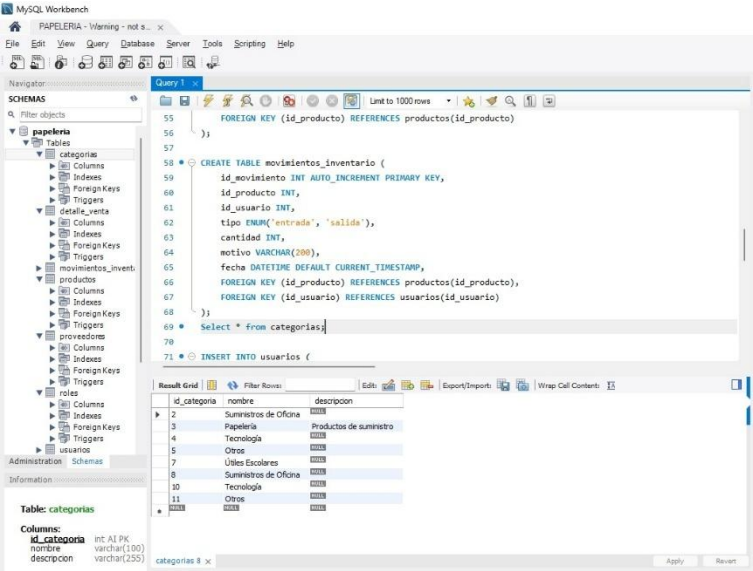
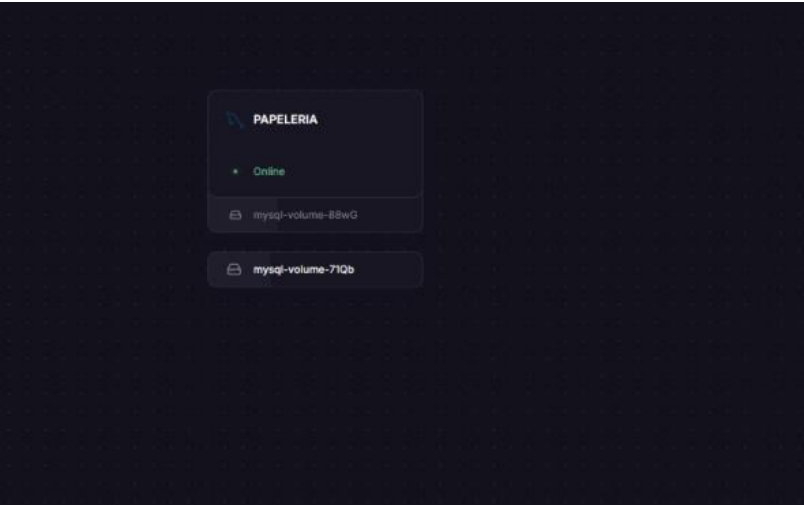
Finalmente, se recomienda incorporar pruebas unitarias utilizando JUnit para validar el funcionamiento de las capas DAO y demás componentes del sistema,

asegurando una mayor calidad del software, facilidad de mantenimiento y una respuesta adecuada ante posibles errores de conexión o cambios futuros.

7. BIBLIOGRAFIA

- Oracle. (2025). *JDBC API Documentation*. <https://docs.oracle.com/javase/>
- MySQL. (2025). *MySQL Documentation*. <https://dev.mysql.com/doc/>
- Railway. (2025). *Railway Documentation*. <https://docs.railway.com/>
- Apache Software Foundation. (2025). Apache Maven Project. <https://maven.apache.org/>

8. ANEXOS



MySQL Workbench - PAPELERIA - Warning - not s...

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

▼ papeleria

- Tables
 - categorias
 - detalle_venta
 - movimientos_invent
 - productos
 - proveedores
 - roles
 - usuarios
 - ventas
- Columns
- Indexes
- Foreign Keys
- Triggers
- Views
- Stored Procedures
- Functions

Information

Table: productos

Columns:

- id_producto int AI PK
- codigo varchar(30)
- nombre varchar(100)
- descripcion varchar(255)
- precio decimal(10,2)
- stock int
- estado int
- id_categoria int

Query 1

```

55 FOREIGN KEY (id_producto) REFERENCES productos(id_producto)
56 })
57
58 CREATE TABLE movimientos_inventario (
59   id_movimiento INT AUTO_INCREMENT PRIMARY KEY,
60   id_producto INT,
61   id_usuario INT,
62   tipo ENUM('entrada', 'salida'),
63   cantidad INT,
64   motivo VARCHAR(200),
65   fecha DATETIME DEFAULT CURRENT_TIMESTAMP,
66   FOREIGN KEY (id_producto) REFERENCES productos(id_producto),
67   FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
68 )
69
70 Select * from productos;
71
72 INSERT INTO usuarios (

```

Result Grid

	id_producto	codigo	nombre	descripcion	precio	stock	estado	id_categoria
1	E404	LAPIZ	LAPIZ NORICA		0.75	179	1	3
4	E504	laminas			1.50	185	1	5
5	E104	Hojas papel			0.12	50	1	3

productos 5 x

Apply Revert

MySQL Workbench - PAPELERIA - Warning - not s...

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

▼ papeleria

- Tables
 - categorias
 - detalle_venta
 - movimientos_inventario
 - productos
 - proveedores
 - roles
 - usuarios
 - ventas
- Columns
- Indexes
- Foreign Keys
- Triggers
- Views
- Stored Procedures
- Functions

Information

Table: proveedores

Columns:

- id_proveedor int AI PK
- nombre varchar(100)
- telefono varchar(20)
- correo varchar(100)
- direccion varchar(255)

Query 1

```

55 FOREIGN KEY (id_producto) REFERENCES productos(id_producto)
56 })
57
58 CREATE TABLE movimientos_inventario (
59   id_movimiento INT AUTO_INCREMENT PRIMARY KEY,
60   id_producto INT,
61   id_usuario INT,
62   tipo ENUM('entrada', 'salida'),
63   cantidad INT,
64   motivo VARCHAR(200),
65   fecha DATETIME DEFAULT CURRENT_TIMESTAMP,
66   FOREIGN KEY (id_producto) REFERENCES productos(id_producto),
67   FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
68 )
69
70 Select * from proveedores;
71
72 INSERT INTO usuarios (

```

Result Grid

	id_proveedor	nombre	telefono	correo	direccion
1		Papelesa	09875432106	sdrcps	Guayaquil
2		Norma	0987652665	fdfghjha	San Luis

proveedores 4 x

Apply Revert

MySQL Workbench - PAPELERIA - Warning - not s...

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

▼ papeleria

- Tables
 - categorias
 - detalle_venta
 - movimientos_inventario
 - productos
 - proveedores
 - roles
 - usuarios
 - ventas
- Columns
- Indexes
- Foreign Keys
- Triggers
- Views
- Stored Procedures
- Functions

Information

Table: roles

Columns:

- id_rol int AI PK
- nombre varchar(50)

Query 1

```

55 FOREIGN KEY (id_producto) REFERENCES productos(id_producto)
56 })
57
58 CREATE TABLE movimientos_inventario (
59   id_movimiento INT AUTO_INCREMENT PRIMARY KEY,
60   id_producto INT,
61   id_usuario INT,
62   tipo ENUM('entrada', 'salida'),
63   cantidad INT,
64   motivo VARCHAR(200),
65   fecha DATETIME DEFAULT CURRENT_TIMESTAMP,
66   FOREIGN KEY (id_producto) REFERENCES productos(id_producto),
67   FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
68 )
69
70 Select * from roles;
71
72 INSERT INTO usuarios (

```

Result Grid

	id_rol	nombre
1		Administrador
2		Cajero
3		Bodeguero

roles 3 x

Apply Revert

MySQL Workbench

PAPELERIA - Warning - not s...

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

Tables

categorias

detalle_venta

movimientos_inventario

productos

proveedores

roles

usuarios

ventas

Columns

Indexes

Foreign Keys

Triggers

Views

Stored Procedures

Functions

sys

Query 1

```

55 FOREIGN KEY (id_producto) REFERENCES productos(id_producto)
56 );
57
58 CREATE TABLE movimientos_inventario (
59   id_movimiento INT AUTO_INCREMENT PRIMARY KEY,
60   id_producto INT,
61   id_usuario INT,
62   tipo ENUM('entrada', 'salida'),
63   cantidad INT,
64   motivo VARCHAR(200),
65   fecha DATETIME DEFAULT CURRENT_TIMESTAMP,
66   FOREIGN KEY (id_producto) REFERENCES productos(id_producto),
67   FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
68 );
69
70 Select * from ventas;
71
72 INSERT INTO usuarios (

```

Result Grid

id_venta	fecha	subtotal	iva	total	id_usuario
1	2026-07-21 04:54:58	0.75	0.08	0.83	1

Administration Schemas

Information

Table: ventas

Columns:

Column	Int A1 PK	datetime	decimal(10,2)	decimal(10,2)
id_venta	PK			
fecha				
subtotal				
iva				

ventas 2 x

Apply Revert

MySQL Workbench

PAPELERIA - Warning - not s...

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

Tables

categorias

detalle_venta

movimientos_inventario

productos

proveedores

roles

usuarios

ventas

Columns

Indexes

Foreign Keys

Triggers

Views

Stored Procedures

Functions

sys

Query 1

```

55 FOREIGN KEY (id_producto) REFERENCES productos(id_producto)
56 );
57
58 CREATE TABLE movimientos_inventario (
59   id_movimiento INT AUTO_INCREMENT PRIMARY KEY,
60   id_producto INT,
61   id_usuario INT,
62   tipo ENUM('entrada', 'salida'),
63   cantidad INT,
64   motivo VARCHAR(200),
65   fecha DATETIME DEFAULT CURRENT_TIMESTAMP,
66   FOREIGN KEY (id_producto) REFERENCES productos(id_producto),
67   FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
68 );
69
70 Select * from ventas;
71
72 INSERT INTO usuarios (

```

Result Grid

id_venta	fecha	subtotal	iva	total	id_usuario
1	2026-07-21 04:54:58	0.75	0.08	0.83	1

Administration Schemas

Information

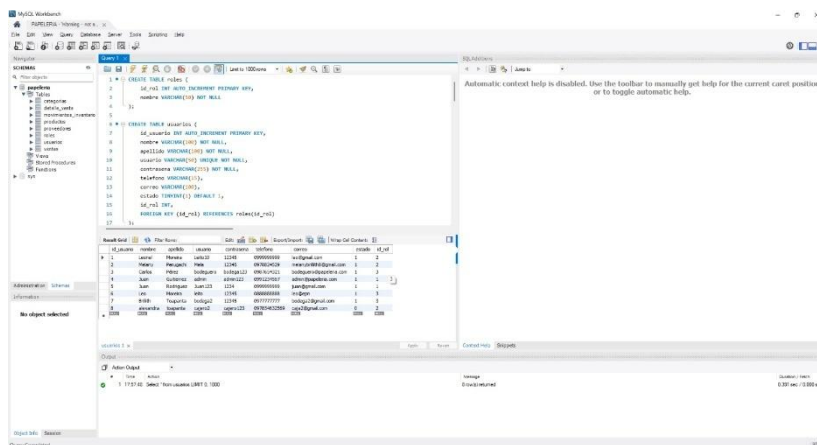
Table: ventas

Columns:

Column	Int A1 PK	datetime	decimal(10,2)	decimal(10,2)
id_venta	PK			
fecha				
subtotal				
iva				

ventas 2 x

Apply Revert



PANEL ADMIN



Usuarios

Productos

Categorías

Proveedores

Ventas

Reportes

Cerrar sesión

Gestión de Usuarios

Mostrar

Nuevo

Editar

Eliminar

Buscar usuario...

Buscar

ID	Nombre	Apellido	Usuario	Teléfono	Correo	Estado	Rol
Tabla sin contenido							

SISTEMA DE PAPELERIA



MÓDULO CAJERO

Registro y Gestión de Ventas

Cerrar Sesión

Buscar Por:

Buscar

Producto: Stock Disponible: Precio (\$):

Cantidad:

+ Agregar al...

Nueva Venta

Eliminar Ite...

Cobrar e I...

Carrito de Venta Actual

Código	Producto	Cantidad	Precio Unitario	Subtotal
Tabla sin contenido				

TOTAL A PAGAR: \$0.00



MÓDULO BODEGUERO

Control de Inventario y Stock

Ver Productos

Stock Bajo

Categorías

Cerrar Sesión

Código

Precio (\$)

Categoría

Descripción

Nombre

Stock

Estado

Nuevo
Guardar
Editar
Eliminar
Entrada Stock
Salida (Mer...
Limpiar

Código	Nombre	Descripción	Precio	Stock	Estado	Categoría
E10H	Hojas papel		0.12	47	Activo	Papelería
E50H	laminas		1.50	184	Activo	Otros
E40H	LAPIZ	LAPIZ NORICA	0.75	9	Activo	Papelería

9. LINK DE VIDEO

<https://drive.google.com/file/d/1k6YVpFvRv4nORDYd0VQmTNwy39-LCsjk/view?usp=sharing>